

FORECASTING

**STREAMGAGE ELEVATIONS AT
USGS NODES**

WWW.THOMASOTT.IO

2026 Update — From Research Prototype to an Automated, Deployable Forecasting Platform



CONTENT

- 01** ABOUT ME
 - 02** THE USE CASE
 - 03** WHY ANOTHER FLOOD PREDICTION PROCESS
 - 04** POMPTON RIVER CROSSING
 - 05** FROM PROTOTYPE TO A PRODUCTION PROBLEM
 - 06** THE FIX: XGBOOST + CONFORMAL CALIBRATION
 - 07** AUTO_PIPELINE.PY — ONE COMMAND, ANY STREAMGAGE
 - 08** PROOF IT GENERALIZES: A SECOND, VERY DIFFERENT RIVER
 - 09** FROM SCRIPT TO PRODUCT: A BROWSER-BASED CONSOLE
 - 10** SHIPPING IT: DOCKER + CONTINUOUS INTEGRATION
 - 11** NEXT STEPS
- 

ABOUT ME

- 30+ years in Civil Engineering, NJ PE, Machine Learning/Data Science expert, dogs love me.

- Engineering consultancy in Northern New Jersey providing Site & Drainage Engineering, Variance Applications, and Green Infrastructure Design.



POMPTON RIVER CROSSING AT ROUTE 23, NEW JERSEY

THE USE CASE

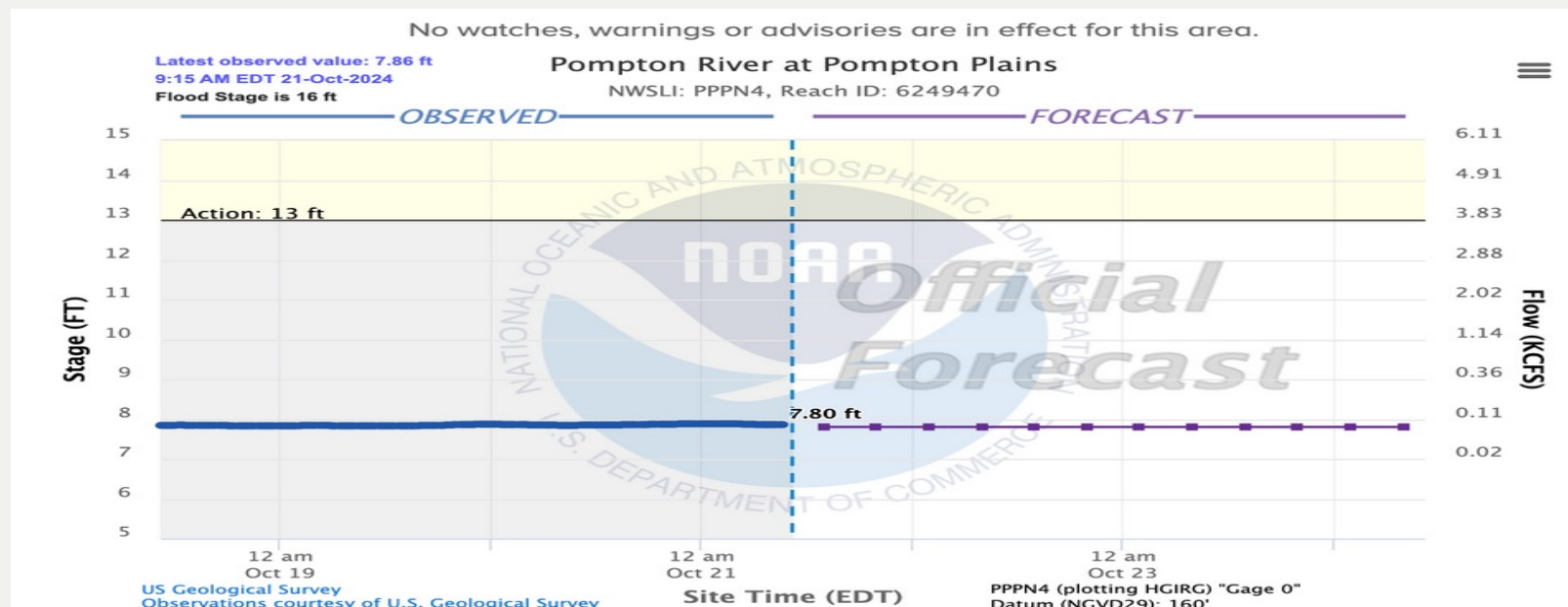


Wayne NJ August 30, 2011 – Hurricane Irene

WHY ANOTHER FLOOD PREDICTION PROCESS

NOAA Streamgage Forecast

- Existing flood stage simulations exist already (NWM)*
- Why another flood prediction process?
- Physics-based vs. data-driven approach

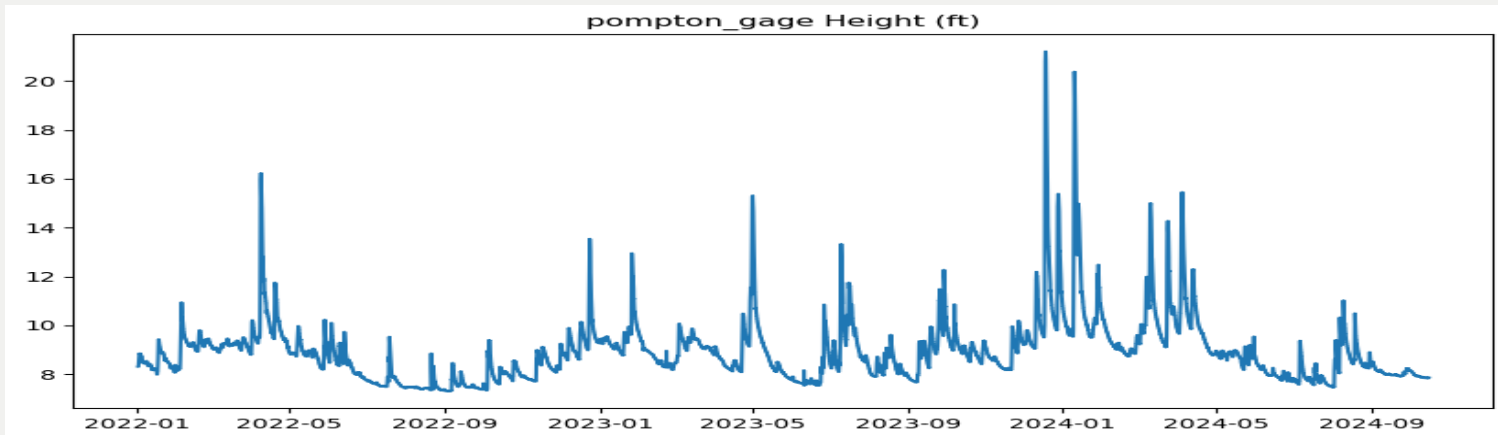


*NWM – National Water Model

WHY ANOTHER FLOOD PREDICTION PROCESS

USGS Streamgage Nodes

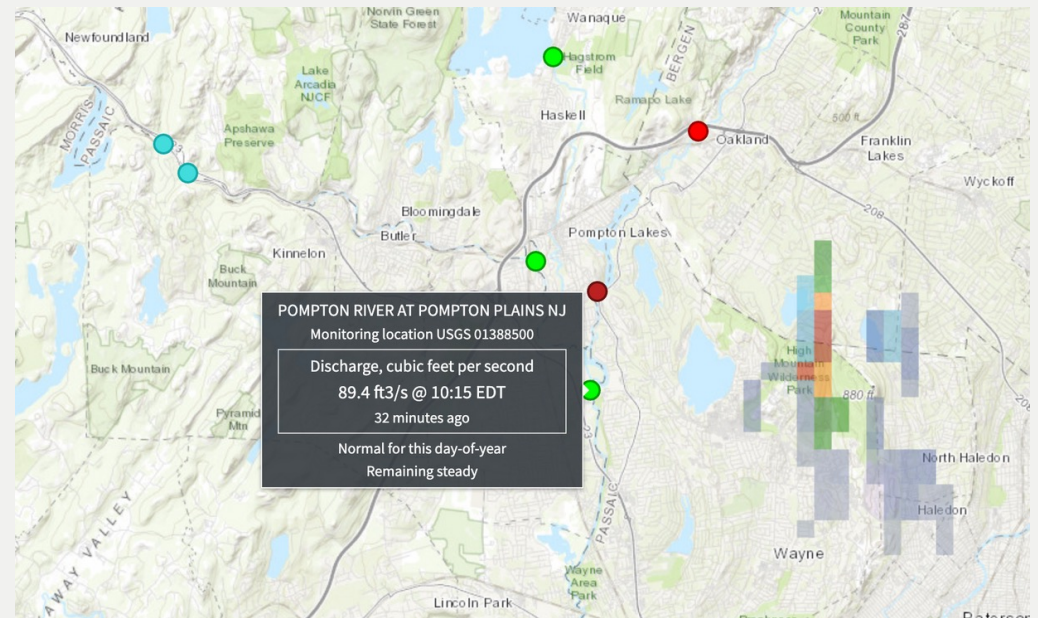
- Why? Apply a data-driven approach for flood predictions
- Explore inferencing at the streamgage node — now automated for any site (see “auto_pipeline.py”)
- Forecast 1, 3, and 6 hour streamgage elevation using machine learning
- Use inferencing results to inform USGS researchers and turn on/off instrumentation remotely



Pompton River gage height, 2022–2024 — the flashy, spike-prone behavior this project targets

POMPTON RIVER CROSSING – WAYNE NJ

- Pompton River crosses at Route 23 in Wayne, NJ
- Confluence of Ramapo & Pequannock Rivers just north of the bridge
- USGS Streamgage #01388500
- Route 23 typically closes at moderate flood stage
- Moderate flood stage elevation = 18 feet



FROM PROTOTYPE TO A PRODUCTION PROBLEM

The 2024 approach

- AutoGluon TimeSeriesPredictor, “fast_training” preset
- Forecast 1 / 3 / 6 / 12 / 24 hours
- Best MAE per horizon: 0.002–0.013 ft (Weighted Ensemble / Theta)

What broke in production

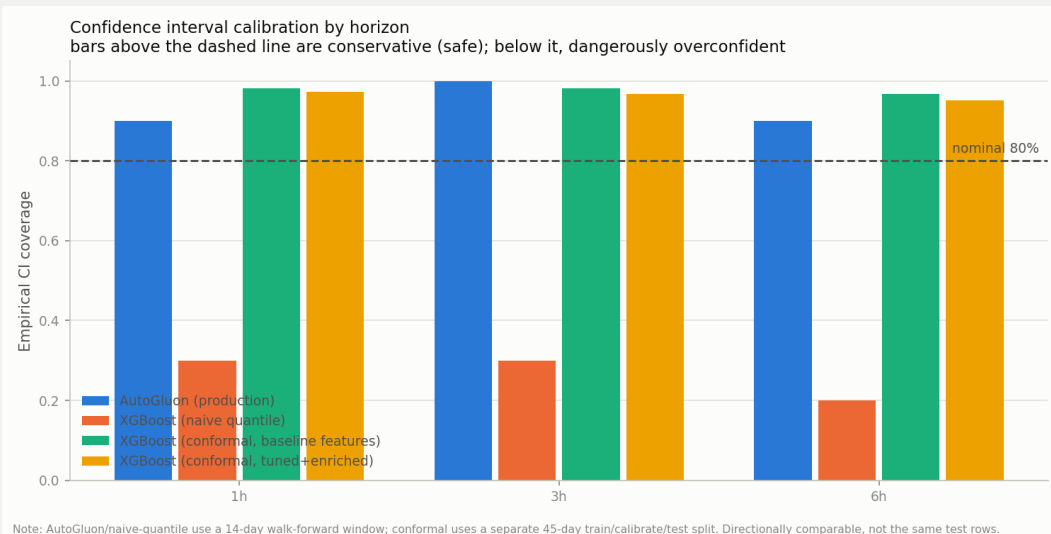
- Saved models are Python pickle files
- Pickle blobs cache numba-JIT'd code paths tied to exact library versions
- A routine Python/numba upgrade left production unable to unpickle the saved predictor
- Accuracy was never the problem — durability was

The fix couldn't just be “retrain” — it had to be a model format that survives version upgrades by design.

THE FIX: XGBOOST + CONFORMAL CALIBRATION

Two problems, two fixes

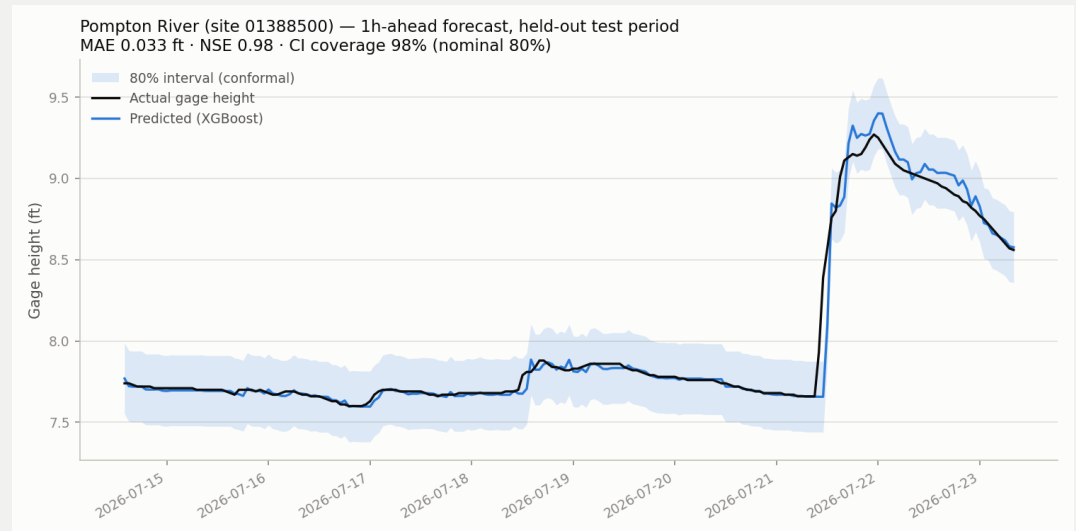
- Portability → XGBoost's native JSON `save_model()` format — a stable, documented schema, not a version-pinned pickle
- Calibration → split conformal prediction instead of naive quantile regression



Naive quantile regression dangerously under-covers (as low as ~20–30%); conformal calibration actually delivers its stated 80% confidence level.

AUTO_PIPELINE.PY — ONE COMMAND, ANY STREAMGAGE

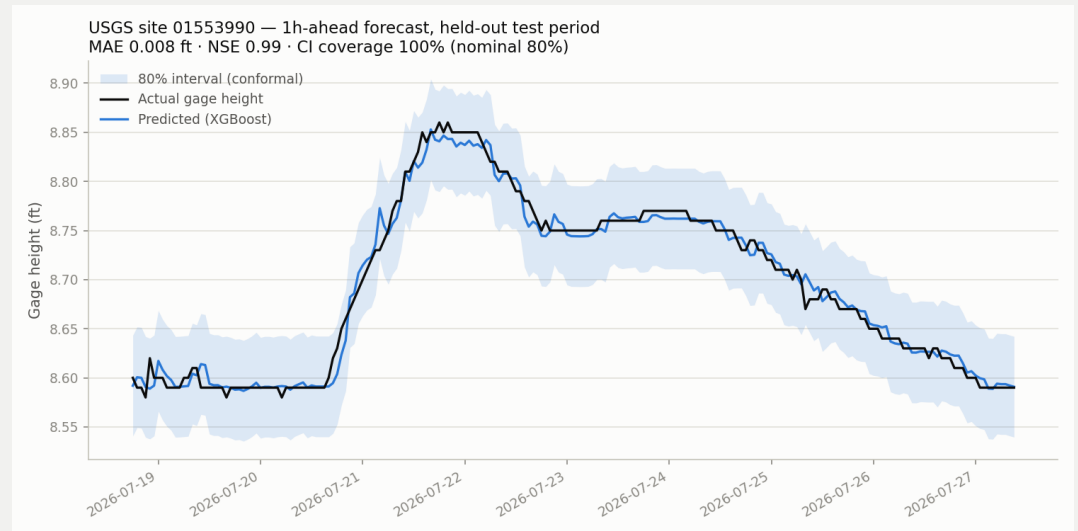
- `python auto_pipeline.py <site_code>` — no more per-site manual trial and error
- Discovers genuinely upstream gages via USGS's NLDI network API (crosses tributary boundaries)
- Auto-selects baseline vs. upstream+weather-enriched+tuned features, per horizon, on held-out accuracy
- Refits on the full window, saves `model.json` + `metadata.json` + a manifest



Real output: Pompton River, 1h-ahead — MAE 0.033 ft · NSE 0.98 · 98% CI coverage (80% nominal)

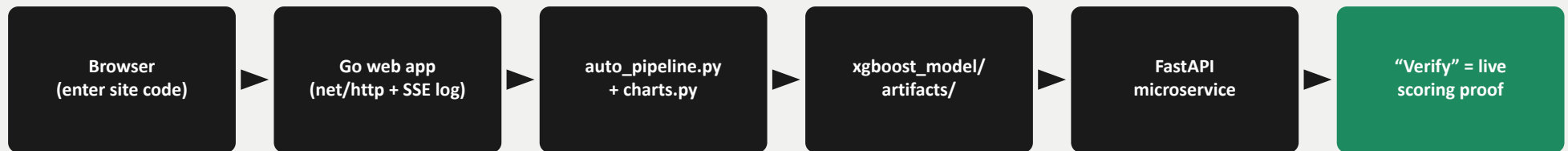
PROOF IT GENERALIZES: A SECOND RIVER

- Tested against USGS 01553990 — Susquehanna River above the dam at Sunbury, PA
- A dam/impoundment gage: reports gage height, never discharge
- `auto_pipeline.py` detected the missing discharge data and fell back to gage-height-only features automatically — no code change, no manual intervention
- **Result: MAE 0.008 ft · NSE 0.99 · 100% CI coverage**



Real output: Susquehanna above Sunbury dam, 1h-ahead forecast

FROM SCRIPT TO PRODUCT: A BROWSER CONSOLE



- Enter a USGS site code, watch `auto_pipeline.py` train and `charts.py` render live over Server-Sent Events — no more staring at a terminal
- Click “Verify” to call the microservice’s own routes and prove the new model is really being served, not just that a file exists
- Zero external dependencies: Go standard library only, vanilla JavaScript, no framework, no CDN

SHIPPING IT: ONE CONTAINER, FOUR CHECKS

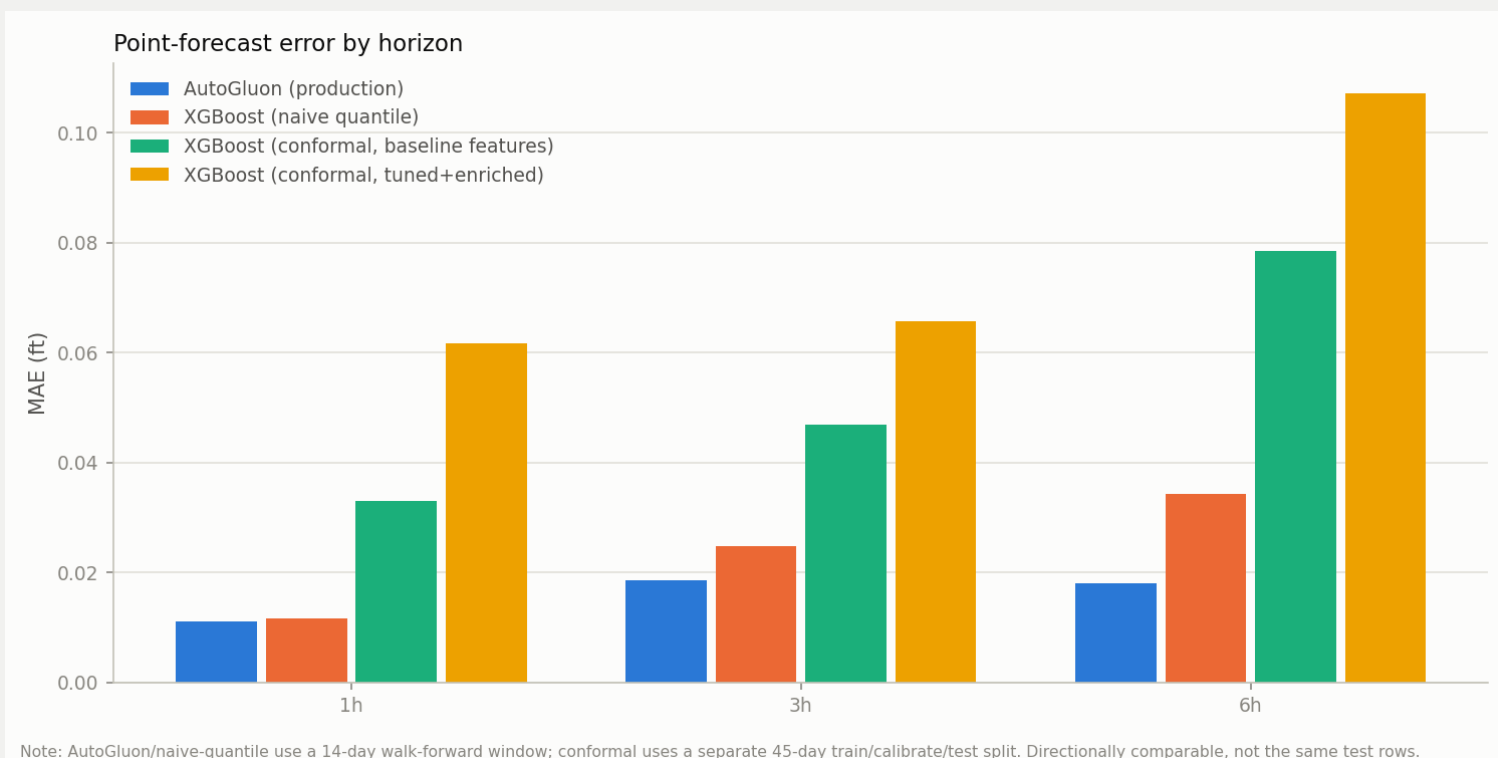
One Docker image

- Multi-stage build: Go binary + Python venv + FastAPI service
- `docker build -t usgs-flood-webapp .`
- `docker run -p 8080:8080 -p 8000:8000 usgs-flood-webapp`
- Both services start together, stop together

Four GitHub Actions workflows, every push/PR

- `microservice-tests.yml` — pytest, fully mocked
- `webapp-tests.yml` — `gofmt` / `vet` / `build` / `test -race`
- `docker-build.yml` — builds the image, smoke-tests live predictions
- `auto-pipeline-tests.yml` — trains a real site against live USGS/NLDI/OpenMeteo APIs

RESULTS: ACCURACY VS. RELIABILITY



AutoGluon's raw accuracy was never the issue. XGBoost + conformal trades a little point-forecast MAE for a model format that survives a Python upgrade and a confidence interval that actually holds its stated coverage.

NEXT STEPS

Shipped since the last version

- ✓ Auto-training templates for any streamgage (`auto_pipeline.py`)
- ✓ Browser-based training + inference console
- ✓ One-command Docker deployment
- ✓ Continuous integration on every push/PR

What's next

- Validate memory/CPU footprint on Raspberry Pi / low-power edge hardware
- Expand auto-training across more USGS watersheds beyond NJ/PA
- Feed “Verify” results back into USGS researcher tooling for remote instrumentation control
- Revisit longer horizons (12h+) only where NSE actually supports it

THANK YOU

Q&A

Contact:

Thomas N. Ott, PE

phone: 973-750-8877

email: info@thomasott.io

